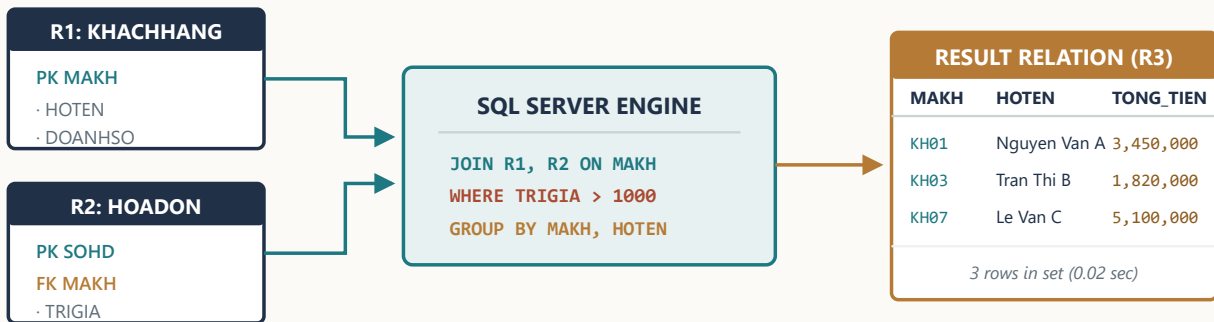


IT004

THỰC HÀNH CƠ SỞ DỮ LIỆU

BIÊN SOẠN: VÕ TRỌNG PHÚC



Thông tin biên soạn

Biên soạn và hệ thống hóa: Võ Trọng Phúc

Môn học: IT004 — Cơ sở dữ liệu (Phần Thực hành T-SQL & SQL Server)

Đối tượng phục vụ: Sinh viên theo học chương trình Cử nhân Công nghệ Thông tin, Hệ thống Thông tin, Kỹ thuật Phần mềm và các ngành kỹ thuật liên quan.

Mục đích & Phương pháp tiếp cận

Cẩm nang Thực hành là ấn phẩm đồng hành cùng *Sổ tay lý thuyết IT004*, tập trung chuyển hóa các mô hình toán học (Đại số quan hệ, Ràng buộc toàn vẹn, Phụ thuộc hàm) thành mã thực thi Transact-SQL (T-SQL) trên hệ quản trị cơ sở dữ liệu Microsoft SQL Server.

Tài liệu xây dựng phương pháp tiếp cận dựa trên 3 trụ cột kỹ thuật:

- Thực nghiệm kiểm chứng (Empirical Verification):** Mỗi câu lệnh đều đi kèm lược đồ đầu vào, phân tích dòng thực thi (Execution Trace) và bảng kết quả tương ứng.
- Tư duy dạng tập hợp (Set-Based Reasoning):** Triệt để giải thích và loại bỏ các thói quen lập trình duyệt con trỏ/vô hướng không tối ưu trên SQL Server.
- Kỹ thuật Trigger an toàn đa dòng (Multi-Row Safety):** Thiết kế các giải pháp ràng buộc dữ liệu đảm bảo toàn vẹn ngay cả khi thực hiện thao tác DML hàng loạt.

Căn cứ học thuật & Tiêu chuẩn

Nội dung cẩm nang được xây dựng và đối chiếu chuẩn xác với:

- Tài liệu kỹ thuật chính thức **Microsoft Learn T-SQL Reference** (các mã nguồn định danh TECH-A01 đến TECH-A11).
- Bộ bài ng chứng khảo sát thực nghiệm **Phase A**: gồm 17 đề thi tự luận/trắc nghiệm, 10 đề thi thực hành máy chính thức và tiến trình 4 bài thực hành Lab 01–04.

Lưu ý về phạm vi và bản quyền

Mọi ví dụ, câu hỏi và lời giải trong tài liệu được biên soạn độc lập, chuẩn hóa cú pháp và bổ sung phân tích cơ chế thực thi chi tiết. Tài liệu không sao chép nguyên văn các bản giải thô của sinh viên.

Kiến trúc Cẩm nang Thực hành

Toàn bộ 8 phần chuyên đề và phụ lục tra cứu chuẩn hóa cho thực hành SQL Server và phòng thi.

Phần 0: Môi trường & Quy trình SSMS 0.1 Cấu trúc SQL Server & SSMS Tr. 4 0.2 Khối lệnh GO & Phạm vi biến Tr. 4 0.3 Checklist 60 giây trước Execute Tr. 4 0.4 Cô lập giao dịch kiểm thử Tr. 4	Phần 1: Xây dựng CSDL với DDL 1.1 CREATE TABLE, PK, FK Tr. 5 1.2 CHECK, DEFAULT, Kiểu dữ liệu Tr. 5 1.3 Tràn số & Bẫy Unicode Tr. 5 1.4 Thứ tự chạy Script DDL Tr. 5
Phần 2: Thao tác Dữ liệu với DML 2.1 INSERT đơn & đa dòng Tr. 6 2.2 UPDATE có tính toán & Subquery Tr. 6 2.3 DELETE an toàn & Khóa ngoại Tr. 6 2.4 Quy trình kiểm chứng 3 bước Tr. 6	Phần 3: Truy vấn Nền tảng & NULL 3.1 SELECT, WHERE, LIKE, BETWEEN Tr. 7 3.2 Logic 3 giá trị & Bẫy IS NULL Tr. 7 3.3 Sắp xếp kết quả ORDER BY Tr. 7
Phần 4: Truy vấn Kết nối & Gom nhóm 4.1 Phép kết INNER & OUTER JOIN Tr. 8 4.2 Dòng thực thi (Execution Trace) Tr. 8 4.3 Self-Join & GROUP BY Tr. 9 4.4 HAVING & Xếp hạng RANK() Tr. 10	Phần 5: Truy vấn Nâng cao & Tập hợp 5.1 Subquery & Toán tử EXISTS Tr. 11 5.2 Bài toán "Tất cả" (NOT EXISTS) Tr. 12 5.3 COUNT(DISTINCT) & Tập chia rỗng Tr. 12 5.4 Phép toán tập hợp (Set Operators) Tr. 13
Phần 6: Ràng buộc & Trigger T-SQL 6.1 Bảng ảo inserted & deleted Tr. 14 6.2 Trigger an toàn đa dòng Tr. 15 6.3 Phân tích sập logic biến vô hướng Tr. 15 6.4 Hiện thực Bảng tầm ảnh hưởng Tr. 16	Phần 7: Hệ thống Chẩn đoán & Gỡ lỗi 7.1 Khung chẩn đoán 5 bước Tr. 17 7.2 Tra cứu 6 nhóm mã lỗi chính Tr. 17
Phần 8: Tiến trình Lab & Phòng thi 8.1 Tiến trình Lab 01 (DDL & RBTV) Tr. 18 8.2 Tiến trình Lab 03–04 (Nâng cao) Tr. 19 8.3 Chiến lược phòng thi thực hành Tr. 20 8.4 Checklist nộp bài & Barem điểm Tr. 20	Phụ lục: Bản đồ Nguồn Microsoft Đối chiếu mã nguồn TECH-A01–A11 Tr. 21 Bảng tra cứu thuật ngữ chuẩn T-SQL Tr. 21

0. Môi trường làm việc & Quy trình chạy Script SSMS

SQL Server Management Studio (SSMS) là công cụ giao diện đồ họa chính thức kết nối và gửi lệnh T-SQL tới SQL Server Database Engine. Hiểu rõ quy trình phân tích cú pháp, phân tách khối lệnh và quản lý giao dịch giúp loại bỏ hoàn toàn các lỗi vận hành thường gặp trong phòng thực hành và phòng thí.

0.1 Phân tách khối lệnh với GO

GO không phải là một câu lệnh T-SQL, mà là một từ khóa điều khiển lệnh (command separator) của SSMS để báo hiệu kết thúc một đợt lệnh (batch) và gửi toàn bộ đợt đó về máy chủ để thực thi.

Phạm vi của biến cục bộ

Mọi biến cục bộ khai báo bằng DECLARE @x ... chỉ tồn tại trong phạm vi của **đợt lệnh** đó. Khi gặp lệnh GO, toàn bộ biến cục bộ trước đó sẽ bị hủy khỏi bộ nhớ.

```
DECLARE @MaKhoa VARCHAR(5) = 'CNTT';
SELECT * FROM GIAOVIEN WHERE MAKHOA = @MaKhoa;
GO -- Kết thúc Batch 1, @MaKhoa bị hủy

-- Lỗi Msg 137: Phải khai báo lại biến!
SELECT * FROM KHOA WHERE MAKHOA = @MaKhoa;
```

0.2 Checklist 60 giây trước khi bấm Execute (F5)

Hạng mục	Nguy cơ tiềm ẩn	Thao tác chuẩn mực
1. Ngăn cảnh CSDL	Vô tình tạo bảng/chèn dữ liệu vào CSDL hệ thống master.	Luôn đặt lệnh USE TenDatabase; GO ở đầu script.
2. Định dạng ngày tháng	Lỗi chuyển đổi kiểu dữ liệu ngày khi hệ thống dùng định dạng Mỹ hoặc Anh.	Thiết lập SET DATEFORMAT YMD; để chuỗi 'YYYY-MM-DD' luôn hợp lệ.
3. Vùng chọn thực thi	Bôi đen thiếu câu lệnh khiến SSMS chỉ chạy một đoạn cắt gây lỗi cú pháp.	Bấm chuột ra ngoài hoặc nhấn Ctrl+A trước khi nhấn F5.
4. Kiểm tra cú pháp	Chạy trực tiếp câu lệnh lỗi có thể làm sai lệch dữ liệu thử nghiệm.	Nhấn Ctrl+F5 (Parse) để kiểm tra cú pháp trước khi chạy.

0.3 Kỹ thuật cô lập giao dịch kiểm thử (Safe Sandbox)

Khi viết các câu lệnh sửa đổi dữ liệu (INSERT, UPDATE, DELETE) hoặc câu lệnh tạo Trigger phức tạp, hãy luôn bao bọc mã trong khối giao dịch có lệnh ROLLBACK để thử nghiệm kết quả mà không làm biến đổi trạng thái bảng gốc:

```
BEGIN TRANSACTION;
-- 1. Xem dữ liệu trước khi thay đổi
SELECT MANV, HOTEN, MUCLUONG FROM GIAOVIEN WHERE MAKHOA = 'CNTT';

-- 2. Thực hiện câu lệnh DML cần kiểm thử
UPDATE GIAOVIEN SET MUCLUONG = MUCLUONG * 1.1 WHERE MAKHOA = 'CNTT';

-- 3. Xem kết quả đã thay đổi trong giao dịch
SELECT MANV, HOTEN, MUCLUONG FROM GIAOVIEN WHERE MAKHOA = 'CNTT';

ROLLBACK TRANSACTION; -- Hoàn tác toàn bộ, dữ liệu trở về nguyên trạng an toàn
```

1. Xây dựng CSDL & Ràng buộc Khai báo (DDL)

Ngôn ngữ định nghĩa dữ liệu (Data Definition Language - DDL) dùng để khởi tạo cấu trúc lưu trữ và các ràng buộc dữ liệu. Thiết kế DDL đúng ngay từ đầu giúp hệ thống tự động ngăn chặn dữ liệu rác ở tầng máy chủ.

1.1 Mini-Lab: Thiết kế bảng chuẩn mực

Xét hai quan hệ `KHACHHANG` và `SANPHAM` trong lược đồ Quản lý bán hàng chuẩn (căn cứ `TECH-A01`, `TECH-A03`):

```
CREATE DATABASE QuanLyBanHang;
GO
USE QuanLyBanHang;
GO

CREATE TABLE KHACHHANG (
    MAKH      CHAR(4)      NOT NULL,
    HOTEN     NVARCHAR(40) NOT NULL,
    DCHI      NVARCHAR(50),
    SODT      VARCHAR(20),
    NGSINH     SMALLDATETIME,
    NGDK      SMALLDATETIME,
    DOANHISO  MONEY        DEFAULT 0,
    CONSTRAINT PK_KHACHHANG PRIMARY KEY (MAKH),
    CONSTRAINT CK_KHACHHANG_NGAY CHECK (NGDK > NGSINH)
);

CREATE TABLE SANPHAM (
    MASP      CHAR(4)      NOT NULL,
    TENSP     NVARCHAR(40) NOT NULL,
    DVT       NVARCHAR(20),
    NUOCSX    NVARCHAR(40) DEFAULT N'Viet Nam',
    GIA       MONEY        CONSTRAINT CK_SANPHAM_GIA CHECK (GIA >= 500),
    CONSTRAINT PK_SANPHAM PRIMARY KEY (MASP)
);
```

Bẫy kiểu dữ liệu: NUMERIC(4,2)

Khi lưu trữ điểm số (thang 10), khai báo `NUMERIC(2,2)` chỉ cho phép giá trị tối đa là `0.99`. Để lưu được điểm `10.00`, bắt buộc phải khai báo `NUMERIC(4,2)` (tổng 4 chữ số, trong đó 2 chữ số phần thập phân).

Thứ tự thực thi DDL

Tạo bảng: Bảng cha (chứa PK) phải tạo trước bảng con (chứa FK).

Xóa bảng: Bảng con (chứa FK) phải xóa trước bảng cha; hoặc xóa FK constraint trước bằng `ALTER TABLE DROP CONSTRAINT`.

2. Thao tác Dữ liệu & Quy trình Kiểm chứng (DML)

Ngôn ngữ thao tác dữ liệu (Data Manipulation Language - DML) bao gồm INSERT , UPDATE , DELETE . Để đảm bảo tính chính xác, mọi thao tác DML cần được thực hiện qua quy trình kiểm chứng 3 bước.

2.1 Mini-Lab: Thao tác dữ liệu mẫu chuẩn

```
-- 1. INSERT đa dòng (Multi-row Value List)
INSERT INTO SANPHAM (MASP, TENSP, DVT, NUOCSX, GIA) VALUES
('BC01', N'But chi 2B', N'Cay', N'Viet Nam', 5000),
('BC02', N'But chi 4B', N'Cay', N'Nhat Ban', 12000),
('ST01', N'So tay A5', N'Quyen', N'Viet Nam', 35000);

-- 2. UPDATE có tính toán và điều kiện Lọc
UPDATE SANPHAM
SET GIA = GIA * 1.05
WHERE NUOCSX = N'Viet Nam' AND DVT = N'Cay';

-- 3. DELETE có điều kiện
DELETE FROM SANPHAM
WHERE GIA < 6000 AND NUOCSX = N'Viet Nam';
```

2.2 Quy trình kiểm chứng 3 bước (Verify Before & After)

Khi làm bài thực hành hoặc bài thi, tuyệt đối không chạy lệnh DML "mù". Hãy luôn kiểm tra tập dòng mục tiêu trước và sau khi thực thi:

QUY TRÌNH KIỂM CHỨNG DML

BƯỚC 1

Kiểm tra dòng sẽ bị tác động: Viết câu lệnh `SELECT * FROM Bang WHERE <DieuKien>` để đếm chính xác số lượng dòng và giá trị cột hiện tại.

BƯỚC 2

Thực thi DML trong Transaction: Bọc trong `BEGIN TRAN ... ROLLBACK TRAN` để xem thông báo hệ thống (N row(s) affected) có khớp với Bước 1 hay không.

BƯỚC 3

Đối chiếu kết quả sau thực thi: Chạy lại câu lệnh `SELECT` để xác nhận giá trị cột đã được cập nhật chính xác theo đúng yêu cầu đề bài.

MASP	TENSP	DVT	NUOCSX	GIA (Trước)	GIA (Sau UPDATE)
BCo1	But chi 2B	Cay	Viet Nam	5,000	5,250 (+5%)
BCo2	But chi 4B	Cay	Nhat Ban	12,000	12,000 (Không đổi)
STo1	So tay A5	Quyen	Viet Nam	35,000	35,000 (Khác DVT)

3. Kỹ thuật Truy vấn Nền tảng & Logic 3 Giá trị

Truy vấn lọc dữ liệu cơ bản trong SQL Server sử dụng mệnh đề `SELECT ... FROM ... WHERE`. Hiểu rõ logic 3 giá trị (True, False, Unknown) là điều kiện tiên quyết để tránh bẫy dữ liệu khi xử lý giá trị `NULL`.

3.1 Cú pháp lọc chuỗi, đoạn & tập hợp

```
-- 1. LIKE: Tìm sản phẩm mã bắt đầu bằng 'B' và tận cùng bằng '01'
SELECT MASP, TENS, GIA FROM SANPHAM WHERE MASP LIKE 'B%01';

-- 2. BETWEEN: Lọc trong đoạn đóng [5000, 35000]
SELECT MASP, TENS, GIA FROM SANPHAM WHERE GIA BETWEEN 5000 AND 35000;

-- 3. IN: Thuộc tập giá trị rời rạc
SELECT MASP, TENS, NUOCSX FROM SANPHAM WHERE NUOCSX IN (N'Việt Nam', N'Nhật Bản');
```

3.2 Logic 3 giá trị (3-Valued Logic) & Bẫy `NULL`

`NULL` thể hiện thông tin chưa biết hoặc không tồn tại, không phải là số 0 hay chuỗi rỗng ''. Mọi phép so sánh bằng toán tử thông thường với `NULL` (như `= NULL` hoặc `!= NULL`) đều trả về `UNKNOWN`, và mệnh đề `WHERE` sẽ loại bỏ tất cả các dòng có kết quả `UNKNOWN`.

Toán hạng A	Toán hạng B	A AND B	A OR B
TRUE	UNKNOWN	UNKNOWN	TRUE
FALSE	UNKNOWN	FALSE	UNKNOWN
UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN

Bắt buộc dùng `IS NULL`

```
-- SAI: Luôn trả về 0 dòng!
SELECT * FROM KHACHHANG WHERE SODT = NULL;

-- ĐÚNG: Cú pháp chuẩn SQL Server
SELECT * FROM KHACHHANG WHERE SODT IS NULL;
```

3.3 Sắp xếp kết quả với `ORDER BY`

`ORDER BY` luôn nằm ở cuối của câu truy vấn. Khi sắp xếp theo thứ tự tăng dần (`ASC`), SQL Server coi `NULL` là giá trị nhỏ nhất và đặt ở đầu kết quả.

4. Phép kết (JOIN) & Dòng thực thi (Execution Trace)

Phép kết kết nối các hàng từ hai hay nhiều bảng dựa trên cột chung. Hiểu rõ dòng thực thi bên trong Database Engine giúp lập trình viên viết truy vấn tối ưu và dự đoán chính xác kết quả đầu ra.

4.1 Worked Example: Truy vấn kết nối nhiều bảng

Yêu cầu: In ra danh sách các hóa đơn mua hàng trong ngày 2024-10-15 gồm: Tên khách hàng, Số hóa đơn và Trị giá, sắp xếp theo Trị giá giảm dần.

```
SELECT KH.HOTEN, HD.SOHD, HD.TRIGIA
FROM HOADON HD
INNER JOIN KHACHHANG KH ON HD.MAKH = KH.MAKH
WHERE HD.NGHD = '2024-10-15'
ORDER BY HD.TRIGIA DESC;
```

4.2 Dòng thực thi chi tiết (Execution Trace)

EXECUTION TRACE — QUÁ TRÌNH XỬ LÝ TRUY VẤN

- 1. FROM & JOIN** Đọc bảng HOADON và kết nối với KHACHHANG trên điều kiện HD.MAKH = KH.MAKH. Tạo tập dòng trung gian chứa đầy đủ cột của cả 2 bảng.
- 2. WHERE** Quét từng dòng trong tập trung gian, áp dụng vị từ lọc HD.NGHD = '2024-10-15'. Các dòng khác ngày lập tức bị loại bỏ khỏi luồng xử lý.
- 3. SELECT** Chiếu (Project) 3 cột cần hiển thị: KH.HOTEN, HD.SOHD, HD.TRIGIA.
- 4. ORDER BY** Sắp xếp tập kết quả cuối cùng theo cột HD.TRIGIA theo thứ tự giảm dần.

4.3 Bảng kết quả mong đợi (Expected Output)

HOTEN	SOHD	TRIGIA	Ghi chú đối chiếu
Nguyen Van A	1002	1,250,000	Khớp MAKH = 'KH01', ngày 2024-10-15
Le Van C	1005	890,000	Khớp MAKH = 'KH07', ngày 2024-10-15
Tran Thi B	1001	320,000	Khớp MAKH = 'KH03', ngày 2024-10-15

INNER JOIN vs LEFT JOIN

INNER JOIN chỉ giữ lại các dòng có khóa khớp ở cả hai bảng. Nếu khách hàng chưa từng có hóa đơn, họ sẽ bị loại. Dùng LEFT OUTER JOIN nếu muốn giữ lại toàn bộ khách hàng và hiển thị NULL ở các cột hóa đơn.

4.4 Kỹ thuật Tự kết (Self-Join) & Gom nhóm (GROUP BY)

Tự kết (Self-Join) là phép kết một bảng với chính nó để giải quyết các mối quan hệ đệ quy (như nhân viên - quản lý). Gom nhóm (`GROUP BY`) phân chia tập dữ liệu thành các nhóm độc lập để tính toán thống kê (căn cứ `TECH-A02`).

4.4.1 Self-Join: Tìm Trưởng bộ môn / Người quản lý

```
-- Lấy tên Giáo viên và Tên Giáo viên quản lý trực tiếp
SELECT GV.MAGV, GV.HOTEN AS TenGiaoVien, QL.HOTEN AS TenNguoiQuanLy
FROM GIAOVIEN GV
LEFT JOIN GIAOVIEN QL ON GV.MAGVQL = QL.MAGV;
```

4.4.2 Cơ chế gom nhóm với `GROUP BY`

Mọi thuộc tính xuất hiện trong danh sách `SELECT` mà không nằm bên trong một hàm kết hợp (như `COUNT` , `SUM` , `AVG` , `MIN` , `MAX`) đều **bắt buộc phải được khai báo trong mệnh đề `GROUP BY`** .

```
-- Thống kê số lượng sản phẩm và giá trung bình theo từng nước sản xuất
SELECT NUOCSX, COUNT(*) AS SoLuongSP, AVG(GIA) AS GiaTrungBinh
FROM SANPHAM
GROUP BY NUOCSX;
```

NUOCSX	SoLuongSP	GiaTrungBinh	Cơ chế xử lý
Viet Nam	12	24,500	Gom toàn bộ 12 dòng có NUOCSX = 'Viet Nam', tính SUM/12
Nhat Ban	6	85,000	Gom toàn bộ 6 dòng có NUOCSX = 'Nhat Ban', tính SUM/6
Singapore	4	110,000	Gom toàn bộ 4 dòng có NUOCSX = 'Singapore', tính SUM/4

Sai lầm kinh điển: Lỗi Msg 8120

Nếu viết `SELECT NUOCSX, TENS, COUNT(*) FROM SANPHAM GROUP BY NUOCSX;` , SQL Server sẽ báo lỗi Msg 8120 vì trong mỗi nhóm `NUOCSX` có nhiều `TENS` khác nhau, máy chủ không thể biết lấy tên nào để hiển thị vào 1 ô kết quả duy nhất.

4.5 Lọc sau gom nhóm (HAVING) & Xếp hạng (RANK)

Sự khác biệt cốt lõi giữa `WHERE` và `HAVING` nằm ở thời điểm bộ lọc được áp dụng trong vòng đời truy vấn: `WHERE` lọc trên từng dòng dữ liệu thô, trong khi `HAVING` lọc trên các nhóm sau khi đã tính toán hàm kết hợp.

4.5.1 Lọc nhóm với `HAVING`

Yêu cầu: Tìm các nước sản xuất có từ 3 sản phẩm trở lên với giá bán tối đa lớn hơn 50,000.

```
SELECT NUOCSX, COUNT(*) AS SoLuongSP, MAX(GIA) AS GiaCaoNhat
FROM SANPHAM
GROUP BY NUOCSX
HAVING COUNT(*) >= 3 AND MAX(GIA) > 50000;
```

4.5.2 Xếp hạng với `RANK() OVER` & `TOP WITH TIES`

Khi cần tìm các đối tượng dẫn đầu bảng xếp hạng (ví dụ: Top 3 khách hàng có doanh số cao nhất), hàm `RANK()` xử lý triệt để trường hợp các đối tượng đồng hạng (cùng giá trị điểm/doanh số).

```
-- 1. Xếp hạng toàn cục bằng RANK()
SELECT MAKH, HOTEN, DOANHSo,
       RANK() OVER (ORDER BY DOANHSo DESC) AS
       Hang
FROM KHACHHANG;
```

```
-- 2. Lấy Top 3 tính cả đồng hạng
SELECT TOP 3 WITH TIES MAKH, HOTEN, DOANHSo
FROM KHACHHANG
ORDER BY DOANHSo DESC;
```

MAKH	HOTEN	DOANHSo	RANK()	TOP 3 WITH TIES	Giải thích cơ chế
KH01	Nguyen Van A	15,000,000	1	Được chọn	Hạng 1 duy nhất
KH04	Pham Van D	12,000,000	2	Được chọn	Hạng 2 đồng hạng (a)
KH09	Hoang Thi E	12,000,000	2	Được chọn	Hạng 2 đồng hạng (b)
KH02	Tran Thi B	9,500,000	4	Được chọn	Hạng 4 (Do đồng hạng trước đó, chiếm vị trí thứ 4 tổng cộng)
KH07	Le Van C	8,000,000	5	Bị loại	Vượt quá ngưỡng Top 3

5. Truy vấn Lồng (Subquery) & Toán tử EXISTS

Truy vấn lồng là câu truy vấn `SELECT` được đặt bên trong một câu lệnh SQL khác. Phân biệt rõ truy vấn con độc lập và truy vấn con tương quan (Correlated Subquery) là chìa khóa để giải quyết các bài toán logic phức tạp (căn cứ `TECH-A07`).

5.1 Truy vấn con độc lập vs Truy vấn con tương quan

Độc lập (Self-Contained)

Chạy duy nhất 1 lần độc lập, trả về giá trị hoặc tập giá trị cho câu cha sử dụng (ví dụ: tìm sản phẩm có giá bán cao nhất).

```
SELECT MASP, TENSP, GIA
FROM SANPHAM
WHERE GIA = (
    SELECT MAX(GIA) FROM SANPHAM
);
```

Tương quan (Correlated)

Chạy lặp lại cho từng dòng của câu cha vì tham chiếu trực tiếp đến cột ở bảng ngoài cùng (như biểu thức lồng `for`).

```
SELECT SP1.MASP, SP1.TENSP, SP1.GIA
FROM SANPHAM SP1
WHERE SP1.GIA = (
    SELECT MAX(SP2.GIA) FROM SANPHAM SP2
    WHERE SP2.NUOC SX = SP1.NUOC SX
);
```

5.2 Toán tử EXISTS / NOT EXISTS (Kiểm tra sự tồn tại)

Toán tử `EXISTS` trả về `TRUE` ngay khi truy vấn con tìm thấy ít nhất một dòng thỏa mãn (cơ chế Short-Circuit Evaluation), không quan tâm danh sách cột trong `SELECT` của câu con (thường viết chuẩn là `SELECT 1`).

```
-- Tìm các sản phẩm chưa từng được bán trong bất kỳ hóa đơn nào
SELECT SP.MASP, SP.TENSP
FROM SANPHAM SP
WHERE NOT EXISTS (
    SELECT 1
    FROM CTHD CT
    WHERE CT.MASP = SP.MASP -- Tham chiếu tương quan với SP.MASP ở ngoài
);
```

CƠ CHẾ ĐÁNH GIÁ CỦA NOT EXISTS

- DÒNG 1** Xét sản phẩm `BC01`: Câu con quét bảng `CTHD` thấy có dòng (`SOHD: 1001, MASP: BC01`)
\$\\rightarrow\$ `EXISTS` là `TRUE` \$\\rightarrow\$ `NOT EXISTS` là `FALSE` \$\\rightarrow\$ **Loại `BC01`**.
- DÒNG 2** Xét sản phẩm `BC05`: Câu con quét toàn bộ `CTHD` không có dòng nào chứa `BC05` \$\\rightarrow\$
`EXISTS` là `FALSE` \$\\rightarrow\$ `NOT EXISTS` là `TRUE` \$\\rightarrow\$ **Giữ lại `BC05`**.

5.3 Bài toán "Tất Cả" & Phép chia trong T-SQL

Bài toán phổ quát yêu cầu tìm các đối tượng bao trọn toàn bộ một tập con dữ liệu (ví dụ: "Tìm khách hàng đã mua **TẤT CẢ** các sản phẩm do Singapore sản xuất"). Đây là dạng bài trọng tâm xuất hiện liên tục trong các đề thi (căn cứ TECH-A07).

5.3.1 Phương pháp chuẩn tắc: Double NOT EXISTS

Biến đổi logic: "Tìm khách hàng sao cho **KHÔNG TỒN TẠI** sản phẩm Singapore nào mà khách hàng đó **CHƯA MUA**."

```
SELECT KH.MAKH, KH.HOTEN
FROM KHACHHANG KH
WHERE NOT EXISTS (
    -- Tập chia: Quét toàn bộ sản phẩm do Singapore sản xuất
    SELECT * FROM SANPHAM SP
    WHERE SP.NUOCSX = 'Singapore'
    AND NOT EXISTS (
        -- Kiểm tra: Khách hàng KH đang xét có mua sản phẩm SP này không?
        SELECT * FROM CTHD CT
        INNER JOIN HOADON HD ON CT.SOHD = HD.SOHD
        WHERE HD.MAKH = KH.MAKH AND CT.MASP = SP.MASP
    )
);
```

DÒNG THỰC THI (CORRELATED TRACE)

1. CÂU NGOÀI

Duyệt từng ứng viên khách hàng $x \in \text{KHACHHANG}$.

2. CÂU GIỮA

Quét tập các sản phẩm yêu cầu $y \in \text{SANPHAM}$ (có `NUOCSX = 'Singapore'`).

3. CÂU TRONG

Kiểm tra xem cặp (x, y) có xuất hiện trong lịch sử mua hàng không. Nếu có ít nhất một yy mà xx chưa mua, câu giữa có kết quả $\rightarrow$ câu ngoài bị loại.

5.3.2 Phương pháp so sánh số lượng: COUNT DISTINCT

```
SELECT HD.MAKH
FROM HOADON HD
INNER JOIN CTHD CT ON HD.SOHD = CT.SOHD
INNER JOIN SANPHAM SP ON CT.MASP = SP.MASP
WHERE SP.NUOCSX = 'Singapore'
GROUP BY HD.MAKH
HAVING COUNT(DISTINCT CT.MASP) = (
    SELECT COUNT(*) FROM SANPHAM WHERE NUOCSX = 'Singapore'
);
```

Xử lý trường hợp tập chia rỗng

Nếu CSDL không có sản phẩm Singapore nào: `Double NOT EXISTS` trả về **toàn bộ khách hàng** (đúng theo ngữ nghĩa toán học logic mệnh đề chân lý rỗng); trong khi phương pháp `COUNT DISTINCT` ở trên trả về 0 dòng do bảng kết nối bị rỗng trước khi gom nhóm.

5.4 Các Phép toán Tập hợp (Set Operators)

SQL Server hỗ trợ ba phép toán tập hợp chuẩn tắ c: `UNION` (Hợp), `EXCEPT` (Trừ) và `INTERSECT` (Giao). Để thực hiện các phép toán này, hai tập truy vấ n phải thỏa mắ n **điề u kiện khả hợp (Union-Compatibility)** (cắ n cứ `TECH-A08`).

5.4.1 Điều kiện khả hợp trong T-SQL

- Hai câu lệnh `SELECT` phải có **cùng số lượng cột**.
- Các cột tương ứng ở cùng vị trí phải có **kiểu dữ liệu tương thích** (hoặc chuyển đổi ngắ m định được).
- Tên cột của tập kắ t quả cuối cùng được lắ y theo **tên cột của câu `SELECT` đầ u tiên**.

5.4.2 Phân biệt UNION vs UNION ALL

```
-- UNION: Tự động Loại bỏ dòng trùng (Chậm hơn)
SELECT MAKH FROM HOADON WHERE YEAR(NGHD) = 2024
UNION
SELECT MAKH FROM KHACHHANG WHERE DOANHSON > 10000000;
```

```
-- UNION ALL: Giữ nguyên toàn bộ dòng (Nhanh hơn)
SELECT MASP FROM CTHD WHERE SOHD = 1001
UNION ALL
SELECT MASP FROM CTHD WHERE SOHD = 1002;
```

5.4.3 Phép trừ (EXCEPT) & Phép giao (INTERSECT)

Yêu cầu: In ra mã các sản phẩm có hóa đơn mua trong năm 2023 nhưng chưa từng được mua trong năm 2024 .

```
SELECT CT.MASP
FROM CTHD CT INNER JOIN HOADON HD ON CT.SOHD = HD.SOHD
WHERE YEAR(HD.NGHD) = 2023

EXCEPT

SELECT CT.MASP
FROM CTHD CT INNER JOIN HOADON HD ON CT.SOHD = HD.SOHD
WHERE YEAR(HD.NGHD) = 2024;
```

Tập truy vấn 1 (Năm 2023)	Tập truy vấn 2 (Năm 2024)	Toán tử	Kết quả trả về
{ BCo1, BCo2, ST01, ST02 }	{ BCo1, ST01, BBo1 }	EXCEPT	{ BCo2, ST02 } (Chỉ có trong 2023)
{ BCo1, BCo2, ST01, ST02 }	{ BCo1, ST01, BBo1 }	INTERSECT	{ BCo1, ST01 } (Có ở cả 2 năm)

6. Ràng buộc Khai báo & Cơ chế DML Trigger

Khi ràng buộc toàn vẹn liên quan đến nhiều bảng hoặc đòi hỏi phép kiểm tra điều kiện phức tạp mà mệnh đề `CHECK` khai báo không thể đảm nhiệm, lập trình viên bắt buộc phải sử dụng **DML Trigger** (căn cứ `TECH-A04`, `TECH-A05`).

6.1 Bảng so sánh phương án hiện thực RBTV

Phương án	Phạm vi áp dụng	Ưu / Nhược điểm
Khai báo (Declarative: CHECK, FK)	Ràng buộc miền giá trị trên 1 bảng; quan hệ khóa chính – khóa ngoại giữa 2 bảng.	Thực thi ở tầng nhân, tốc độ cao nhất; không thể chứa subquery hoặc tham chiếu bảng ngoài.
Thủ tục (Procedural: AFTER Trigger)	Ràng buộc liên bảng, kiểm tra logic nghiệp vụ phức tạp, ràng buộc so sánh ngày tháng giữa các bảng.	Linh hoạt tối đa; yêu cầu thiết kế chuẩn dạng tập hợp để tránh quá tải và lỗi đa dòng.

6.2 Hai bảng ảo chuyển tiếp: `inserted` & `deleted`

Khi sự kiện DML xảy ra, SQL Server tự động tạo hai bảng tạm trong bộ nhớ có cấu trúc cột y hệt bảng gốc (căn cứ `TECH-A05`):

Thao tác DML	Bảng <code>inserted</code> chứa gì?	Bảng <code>deleted</code> chứa gì?
INSERT	Các dòng mới vừa được thêm vào	Rỗng (Không có dòng nào)
DELETE	Rỗng (Không có dòng nào)	Các dòng cũ vừa bị xóa đi
UPDATE	Các dòng mới sau khi sửa	Các dòng cũ trước khi sửa

Bản chất của lệnh UPDATE trong SQL Server

SQL Server thực hiện `UPDATE` bằng cách xóa giá trị cũ (đưa vào `deleted`) và chèn giá trị mới (đưa vào `inserted`). Vì vậy, trong Trigger của `UPDATE`, cả hai bảng ảo đều chứa dữ liệu.

6.2 Thiết kế Trigger An toàn Đa dòng (Multi-Row Safety)

SQL Server là hệ quản trị xử lý theo tập hợp. Một câu lệnh `INSERT`, `UPDATE` hay `DELETE` có thể tác động cùng lúc lên hàng nghìn dòng. Trigger phải luôn được thiết kế để xử lý chính xác tình huống đa dòng này (căn cứ TECH-A06).

6.2.1 Phân tích sai lầm kinh điển: Biến vô hướng (Scalar Trigger)

CÁCH VIẾT SAI (Dễ sập logic)

```
-- SAI LẦM: Dùng biến vô hướng
DECLARE @NgayHD SMALLDATETIME;
SELECT @NgayHD = NGHD FROM inserted;
-- Nếu inserted có 100 dòng,
-- biến @NgayHD chỉ nhận giá trị
-- của 1 dòng ngẫu nhiên duy nhất!
-- 99 dòng vi phạm khác bị bỏ Lọt!
```

CÁCH VIẾT ĐÚNG (Dạng tập hợp)

```
-- CHUẨN MỰC: Kết nối tập hợp
IF EXISTS (
    SELECT 1
    FROM inserted i
    INNER JOIN KHACHHANG kh
        ON i.MAKH = kh.MAKH
    WHERE i.NGHD < kh.NGDK
)
-- Bắt trọn vẹn mọi dòng vi phạm!
```

6.2.2 Mẫu Trigger chuẩn tắc dạng tập hợp

Yêu cầu: Ngày mua hàng (`NGHD`) của một hóa đơn phải lớn hơn hoặc bằng Ngày đăng ký (`NGDK`) của khách hàng đó.

```
CREATE TRIGGER trg_Check_HoaDon_NgayMua
ON HOADON
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- Kiểm tra xem có BẤT KỲ dòng nào trong inserted vi phạm quy tắc hay không
    IF EXISTS (
        SELECT 1
        FROM inserted i
        INNER JOIN KHACHHANG kh ON i.MAKH = kh.MAKH
        WHERE i.NGHD < kh.NGDK
    )
    BEGIN
        RAISERROR (N'Lỗi: Ngày mua hàng (NGHD) không thể trước Ngày đăng ký thành viên (NGDK)!', 16,
1);
        ROLLBACK TRANSACTION; -- Hủy bỏ toàn bộ thao tác DML vi phạm
        RETURN;
    END
END;
GO
```

6.3 Hiện thực Bảng tầm ảnh hưởng trong T-SQL

Bảng tầm ảnh hưởng trong lý thuyết xác định chính xác các sự kiện DML trên từng quan hệ có nguy cơ gây vi phạm. Trong thực hành T-SQL, mỗi quan hệ có dữ liệu + sẽ được cài đặt một Trigger tương ứng.

6.3.1 Phân tích bối cảnh & Bảng tầm ảnh hưởng

Ràng buộc: "Trưởng khoa của một khoa phải là giáo viên thuộc chính khoa đó."

Quan hệ	Thêm (INSERT)	Xóa (DELETE)	Sửa (UPDATE)
KHOA	+ (Cần kiểm tra TRUONGKHOA)	-	+(TRUONGKHOA)
GIAOVIEN	-	+ (Nếu xóa đúng GV là Trưởng khoa)	+(MAKHOA)

6.3.2 Cài đặt Trigger hai chiều

```
-- 1. Trigger trên bảng KHOA (Kiểm tra khi bổ nhiệm/sửa Trưởng khoa)
CREATE TRIGGER trg_Khoa_CheckTruongKhoa
ON KHOA AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (
        SELECT 1 FROM inserted k
        LEFT JOIN GIAOVIEN gv ON k.TRUONGKHOA = gv.MAGV AND k.MAKHOA = gv.MAKHOA
        WHERE k.TRUONGKHOA IS NOT NULL AND gv.MAGV IS NULL
    )
    BEGIN
        RAISERROR (N'Lỗi: Trưởng khoa phải là giáo viên thuộc chính khoa đó!', 16, 1);
        ROLLBACK TRAN;
    END
END;
GO

-- 2. Trigger trên bảng GIAOVIEN (Ngăn chuyển khoa/xóa GV đang làm Trưởng khoa)
CREATE TRIGGER trg_GiaoVien_CheckKhoa
ON GIAOVIEN AFTER DELETE, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (
        SELECT 1 FROM deleted d
        INNER JOIN KHOA k ON d.MAGV = k.TRUONGKHOA AND d.MAKHOA = k.MAKHOA
    )
    BEGIN
        RAISERROR (N'Lỗi: Không thể xóa hoặc đổi khoa của giáo viên đang giữ chức vụ Trưởng khoa!',
16, 1);
        ROLLBACK TRAN;
    END
END;
GO
```


7. Hệ thống Chẩn đoán & Gỡ lỗi Transact-SQL

Khi gặp thông báo lỗi từ SQL Server, áp dụng quy trình chẩn đoán 5 bước chuẩn hóa giúp xác định nguyên nhân gốc rễ và xử lý triệt để trong vòng 60 giây.

Msg 208: Invalid object name '...'

TRIỆU CHỨNG: Câu lệnh không thể tìm thấy bảng hoặc khung nhìn.

NGUYÊN NHÂN: 1) Sai Database (đang ở `master`); 2) Bảng chưa được tạo; 3) Viết sai chính tả tên bảng.

CÁCH SỬA: Chạy lệnh `USE TenDatabase; GO` và kiểm tra lại danh sách bảng trong Object Explorer.

Msg 547: Conflict with FK / CHECK constraint

TRIỆU CHỨNG: Bị từ chối khi chèn/sửa dữ liệu vi phạm ràng buộc.

NGUYÊN NHÂN: 1) FK trỏ tới giá trị PK không tồn tại ở bảng cha; 2) Giá trị không thỏa mãn biểu thức CHECK.

CÁCH SỬA: Kiểm tra bảng cha xem đã có khóa chính đó chưa; kiểm tra lại giá trị cột so với điều kiện CHECK.

Msg 8152 / 8115: Truncation / Arithmetic overflow

TRIỆU CHỨNG: Chuỗi bị cắt ngắn hoặc tràn số học.

NGUYÊN NHÂN: Độ dài chuỗi vượt quá `VARCHAR(N)` hoặc giá trị vượt quá miền `NUMERIC(p,s)` (ví dụ: điểm 10 vào `NUMERIC(2,2)`).

CÁCH SỬA: `ALTER TABLE Bang ALTER COLUMN Cot NUMERIC(4,2);`

Msg 8120: Column invalid in SELECT / GROUP BY

TRIỆU CHỨNG: Không thể thực thi câu lệnh gom nhóm.

NGUYÊN NHÂN: Có cột trong `SELECT` không nằm trong `GROUP BY` và cũng không được bọc trong hàm kết hợp.

CÁCH SỬA: Thêm cột đó vào danh sách `GROUP BY` hoặc bọc bằng `MAX()`, `MIN()`, `COUNT()`.

Msg 512: Subquery returned more than 1 value

TRIỆU CHỨNG: Truy vấn con trả về nhiều hơn 1 dòng khi dùng toán tử vô hướng.

NGUYÊN NHÂN: Dùng toán tử `=`, `>`, `<` với subquery trả về tập nhiều dòng thay vì 1 giá trị duy nhất.

CÁCH SỬA: Thay `=` bằng toán tử tập hợp `IN`, hoặc thêm `TOP 1` / hàm gộp `MAX()` vào câu con.

Msg 245: Conversion failed when converting varchar

TRIỆU CHỨNG: Lỗi ép kiểu dữ liệu chuỗi sang kiểu số hoặc ngày tháng.

NGUYÊN NHÂN: Chuỗi chứa ký tự chữ cái hoặc sai định dạng ngày tháng hệ thống.

CÁCH SỬA: Dùng hàm `TRY_CAST(cot AS INT)` hoặc thiết lập `SET DATEFORMAT YMD;`

8.1 Tiến trình Thực hành Mẫu — Tái dựng Lab 01 (DDL & RBTV)

Dựa trên khảo sát thực nghiệm mã nguồn thực hành Phase A (nguồn LOC-SQL-LAB01 , LAB-QLBH-QLGV-CORPUS), buổi Lab 01 tập trung xây dựng toàn bộ cấu trúc bảng và ràng buộc cho hai cơ sở dữ liệu nên tăng QuanLyBanHang và QuanLyGiaoVu .

8.1.1 Cấu trúc lược đồ Quản lý Giáo vụ (QLGV)

Quan hệ	Danh sách thuộc tính (PK gạch chân, FK in nghiêng)	Ràng buộc toàn vẹn cốt lõi
KHOA	<u>MAKHOA</u> , TENKHOA, NGTLAP, <i>TRUONGKHOA</i>	FK TRUONGKHOA \$\\rightarrow\$ GIAOVIEN(MAGV)
MONHOC	<u>MAMH</u> , TENMH, TCLT, TCTH, <i>MAKHOA</i>	CHECK (TCLT >= 1 AND TCTH >= 0)
DIEUKIEN	<u>MAMH</u> , <i>MAMH_TRUOC</i>	Composite PK; CHECK (MAMH != MAMH_TRUOC)
GIAOVIEN	<u>MAGV</u> , HOTEN, HOCVI, HOCHAM, GIOITINH, NGSINH, NGVL, HESO, MUCLUONG, <i>MAKHOA</i>	CHECK (HOCVI IN ('CN', 'KS', 'ThS', 'TS', 'PTS')) CHECK (HESO > 0 AND MUCLUONG > 0)
LOP	<u>MALOP</u> , TENLOP, <i>TRGLOP</i> , SISO, <i>MAGVCN</i>	CHECK (SISO > 0)
HOCVIEN	<u>MAHV</u> , HO, TEN, NGSINH, GIOITINH, NOISINH, <i>MALOP</i>	CHECK (GIOITINH IN ('Nam', 'Nu'))
GIANGDAY	<u>MALOP</u> , <u>MAMH</u> , <i>MAGV</i> , HOCKY, NAM, TUNGAY, DENNGAY	CHECK (HOCKY IN (1, 2, 3)) CHECK (TUNGAY < DENNGAY)
KETQUATHI	<u>MAHV</u> , <u>MAMH</u> , <u>LANTHI</u> , NGTHI, DIEM, KQUA	DIEM NUMERIC(4,2) CHECK (DIEM BETWEEN 0 AND 10)

8.1.2 Script DDL giải quyết quan hệ vòng (Circular FK Dependency)

Giữa KHOA và GIAOVIEN tồn tại quan hệ tham chiếu vòng: KHOA.TRUONGKHOA trỏ tới GIAOVIEN.MAGV , trong khi GIAOVIEN.MAKHOA trỏ tới KHOA.MAKHOA . Cách giải quyết chuẩn tắc:

```
-- 1. Tạo bảng KHOA không có Foreign Key TRUONGKHOA
CREATE TABLE KHOA (MAKHOA VARCHAR(4) PRIMARY KEY, TENKHOA NVARCHAR(40), TRUONGKHOA CHAR(4));

-- 2. Tạo bảng GIAOVIEN với Foreign Key trỏ về KHOA
CREATE TABLE GIAOVIEN (MAGV CHAR(4) PRIMARY KEY, HOTEN NVARCHAR(40), MAKHOA VARCHAR(4) FOREIGN KEY REFERENCES KHOA(MAKHOA));

-- 3. Bổ sung Foreign Key TRUONGKHOA sau khi cả 2 bảng đã tồn tại
ALTER TABLE KHOA ADD CONSTRAINT FK_KHOA_TRUONGKHOA FOREIGN KEY (TRUONGKHOA) REFERENCES GIAOVIEN(MAGV);
```

8.2 Tiến trình Thực hành Mẫu — Tái dựng Lab 03 & Lab 04 (Nâng cao)

Dựa trên khảo sát thực nghiệm Phase A (nguồn `LOC-SQL-LAB03`, `LOC-SQL-LAB04`), các buổi Lab nâng cao tập trung vào truy vấn thống kê gom nhóm phức tạp, xếp hạng phân vùng và bài toán phép chia.

8.2.1 Bài toán 1: Tìm môn học có điểm trung bình cao nhất theo từng khoa

Kỹ thuật: Sử dụng `RANK() OVER (PARTITION BY ... ORDER BY ...)` (căn cứ `TECH-A02`):

```
WITH BangXepHang AS (
    SELECT mh.MAKHOA, mh.MAMH, mh.TENMH, AVG(kq.DIEM) AS DiemTB,
           RANK() OVER (PARTITION BY mh.MAKHOA ORDER BY AVG(kq.DIEM) DESC) AS Hang
    FROM MONHOC mh
    INNER JOIN KETQUATHI kq ON mh.MAMH = kq.MAMH
    WHERE kq.LANTHI = 1
    GROUP BY mh.MAKHOA, mh.MAMH, mh.TENMH
)
SELECT MAKHOA, MAMH, TENMH, DiemTB
FROM BangXepHang
WHERE Hang = 1;
```

8.2.2 Bài toán 2: Tìm học viên thi đạt tất cả các môn thuộc khoa CNTT phụ trách

Kỹ thuật: Double `NOT EXISTS` kết hợp điều kiện điểm đạt `>= 5.0` (căn cứ `TECH-A07`):

```
SELECT hv.MAHV, hv.HO, hv.TEN
FROM HOCVIEN hv
WHERE NOT EXISTS (
    -- 1. Quét tất cả môn học do khoa CNTT phụ trách
    SELECT * FROM MONHOC mh
    WHERE mh.MAKHOA = 'CNTT'
    AND NOT EXISTS (
        -- 2. Kiểm tra học viên hv đã thi đạt môn mh này chưa?
        SELECT * FROM KETQUATHI kq
        WHERE kq.MAHV = hv.MAHV AND kq.MAMH = mh.MAMH AND kq.DIEM >= 5.0
    )
);
```

Điểm mấu chốt khi làm bài Lab 04

Khi đề bài yêu cầu "lần thi sau cùng", ta phải dùng Subquery lọc `LANTHI = (SELECT MAX(LANTHI) FROM KETQUATHI kq2 WHERE kq2.MAHV = kq1.MAHV AND kq2.MAMH = kq1.MAMH)` thay vì chỉ lọc cứng `LANTHI = 1`.

8.3 Chiến lược & Quy trình Phòng thi Thực hành

Khảo sát từ 10 hiện vật đề thi thực hành chính thức (nguồn PRAC-2024-2025-HK1-01 , PRAC-2023-2024-HK1-FINAL-01 , PRAC-2022-2023-HK1-D03 , UIT-006), các bài thi thực hành máy có thời lượng 60–90 phút với cấu trúc điểm phân bố đều cho 3 phần: Tạo bảng/RBTV (2–3đ), Truy vấn SQL (4–5đ) và Trigger (2–3đ).

8.3.1 Bảng phân bố thời gian chuẩn (Chiến thuật 75 phút)

Giai đoạn	Thời lượng	Nhiệm vụ trọng tâm
1. Setup & Đọc đề	10 phút	Tạo file MSSV_HoTen.sql , tạo Database, xác định toàn bộ PK/FK và vẽ nháp mối quan hệ giữa các bảng.
2. DDL & Ràng buộc	15 phút	Viết CREATE TABLE , khai báo đúng kiểu dữ liệu, PK, FK và các ràng buộc CHECK đơn giản.
3. Nạp dữ liệu mẫu	10 phút	Chèn ít nhất 3–4 dòng mẫu cho mỗi bảng để có dữ liệu thực nghiệm kiểm chứng truy vấn.
4. Viết Truy vấn SQL	25 phút	Giải quyết lần lượt từ câu cơ bản (WHERE, JOIN) đến câu nâng cao (GROUP BY, Subquery, NOT EXISTS).
5. Viết Trigger & Nộp	15 phút	Viết Trigger an toàn đa dòng, chạy thử nghiệm với 1 lệnh INSERT vi phạm, kiểm tra toàn bộ file trước khi nộp.

8.3.2 Checklist 5 điểm trước khi nộp bài

CHECKLIST NỘP BÀI THI THỰC HÀNH	
1. TOÀN CỤC	Nhấn Ctrl+A rồi Ctrl+F5 (Parse) để đảm bảo không có bất kỳ dòng lệnh nào bị lỗi cú pháp làm đứt đoạn quá trình chấm thi.
2. TÁCH BATCH	Đảm bảo có từ khóa GO sau mỗi khối lệnh CREATE TRIGGER hoặc CREATE PROCEDURE .
3. XÓA TẠM	Xóa bỏ các câu lệnh DROP TABLE hoặc DELETE tạm thời dùng trong lúc thử nghiệm.
4. ĐẶT TÊN FILE	Lưu tệp script .sql đúng cấu trúc quy định của hội đồng thi (ví dụ: 23520xxx_De01.sql).

Phụ lục: Bản đồ Tham chiếu Microsoft Learn (TECH-A)

Toàn bộ cơ chế kỹ thuật và cú pháp trong cẩm nang được đồ i chiếu u trực tiế p với tài liệu kỹ thuật chính thức Microsoft Learn (nguồn chuẩn Tier A, căn cứ `research/v1.1_phase_a/source_inventory.md`):

Mã định danh	Chủ đề kỹ thuật Microsoft Learn	Từ khóa T-SQL cốt lõi	Chuyên đề cẩm nang
TECH-A01	CREATE TABLE (Transact-SQL)	TABLE, PK, FK, NOT NULL	Phầ n 1: DDL Khai báo
TECH-A02	SELECT - GROUP BY (Transact-SQL)	GROUP BY, HAVING, COUNT	Phầ n 4: Phân tích tổng hợp
TECH-A03	CREATE CHECK Constraints	CONSTRAINT ... CHECK	Phầ n 1: Ràng buộc miề n
TECH-A04	CREATE TRIGGER (Transact-SQL)	AFTER INSERT, UPDATE	Phầ n 6: Trigger nâng cao
TECH-A05	Use inserted & deleted tables	inserted, deleted	Phầ n 6: Bảng ảo chuyển tiế p
TECH-A06	Handle multiple rows in DML triggers	JOIN inserted, Set-based	Phầ n 6: An toàn đa dòng
TECH-A07	EXISTS (Transact-SQL)	EXISTS, NOT EXISTS	Phầ n 5: Bài toán "Tắ t cả"
TECH-A08	SET Operators - UNION	UNION, EXCEPT, INTERSECT	Phầ n 5: Phép toán tập hợp
TECH-A09	CREATE PROCEDURE (Transact-SQL)	PROCEDURE, EXEC, PARAM	Phụ lục: Stored Procedure
TECH-A10	CREATE VIEW (Transact-SQL)	CREATE VIEW ... AS SELECT	Phụ lục: Khung nhìn View
TECH-A11	CREATE FUNCTION (Transact-SQL)	FUNCTION, RETURNS TABLE	Phụ lục: UDF đọc thêm

Chuẩn mực tra cứu kỹ thuật

Khi phát triển các giải pháp cơ sở dữ liệu quy mô doanh nghiệp hoặc triển khai thực tế ́ trên Azure SQL / SQL Server 2022, sinh viên và kỹ sư nên đồ i chiếu u trực tiế p các mã nguồ n TECH-A01 – TECH-A11 trên cổng tài liệu `learn.microsoft.com` để nắ m bắ t các tính năng tớ i ưu hóa mới nhấ t.